1. What is Spring Security and why use it in modern Spring Boot applications?

Spring Security is a powerful and highly customizable security framework used to protect Spring and Spring Boot applications.

Why use Spring Security in modern Spring Boot applications?

- **Authentication**: It verifies *who the user is* using mechanisms like username/password, JWT, OAuth2, LDAP, or social logins.
- **Authorization**: It controls *what the user is allowed to do* by enforcing role-based and permission-based access (e.g., ROLE_ADMIN, ROLE_USER).
- **Protection against common attacks**: Spring Security provides built-in protection against CSRF, XSS, session fixation, clickjacking, and brute-force attacks.
- **Seamless Spring Boot integration**: It auto-configures security with minimal setup and works naturally with Spring MVC, REST APIs, and Spring Data.
- **Support for modern security standards**: It supports OAuth2, OpenID Connect (OIDC), JWT, and SSO, which are widely used in modern microservices and cloud-native applications.
- **Method-level security**: It allows securing business logic using annotations like @PreAuthorize, @PostAuthorize, and @Secured.
- **Flexible configuration**: Security rules can be configured using Java config, annotations, or custom filters based on application needs.
- **Scalable for microservices**: Works well with stateless authentication (JWT) and API gateways in distributed systems.
- **Industry-trusted framework**: Actively maintained by Spring team and widely adopted in enterprise applications.

In short: Spring Security ensures that your Spring Boot application is **secure, scalable, and compliant with modern security requirements**.

2. Explain the difference between authentication and authorization in Spring Security.

Authentication: Verifies *who the user is* (e.g., username/password, JWT). It happens **first** and confirms the user's identity.

Authorization: Determines *what the authenticated user can access* (roles/permissions like ROLE_ADMIN). It happens **after authentication**.

3. What is the SecurityFilterChain and why is it important?

SecurityFilterChain is a core component in Spring Security that defines a **chain of security filters** applied to incoming HTTP requests.

Why is it important?

- It **intercepts every request** and applies security checks such as authentication, authorization, CSRF protection, and session management before the request reaches controllers.
- It allows **custom security configuration**, where developers define which URLs are secured, which are public, authentication mechanisms (JWT, form login, OAuth2), and access rules.

👉 **In short**: SecurityFilterChain controls **how and in what order security rules are enforced** for every request in a Spring Boot application.

4. what is CSRF token?

A CSRF token (Cross-Site Request Forgery token) is a unique, random, and secret value generated by the server and sent to the client to verify that a request is legitimate.

What it does

- It ensures that state-changing requests (POST, PUT, DELETE) are made intentionally by the authenticated user, not by a malicious website.

How it works

- Server generates a CSRF token and stores it (session or cookie).
- Client sends the token back with each request (header or form field).
- Server validates the token before processing the request.

Why it's important

- Prevents attackers from performing actions on behalf of logged-in users using their browser session.

👉 In short: A CSRF token is a security key that protects web applications from forged requests made without the user's consent.

5. How does Spring Security handle CSRF protection and when should it be disabled (e.g., REST APIs)?

CSRF Protection in Spring Security & When to Disable It

- **How Spring Security handles CSRF**:
  Spring Security enables CSRF protection by default for state-changing requests (POST, PUT, DELETE). It uses a **CSRF token** that must be sent with each request and validated on the server to prevent unauthorized actions.
- **When CSRF should be disabled**:
  CSRF can be disabled for **stateless REST APIs** that use **JWT, OAuth2, or Basic Auth**, because these APIs do not rely on cookies or server-side sessions.

👉 **In short**: Enable CSRF for **browser-based apps with sessions**; disable it for **stateless REST APIs**.

6. What are the common password encoders in Spring Security? Why is BCrypt recommended?

Common Password Encoders in Spring Security

- **NoOpPasswordEncoder** – Stores passwords in plain text (not secure, only for testing).
- **BCryptPasswordEncoder** – Uses the BCrypt hashing algorithm with salt.
- **Pbkdf2PasswordEncoder** – Uses PBKDF2 hashing with configurable iterations.
- **SCryptPasswordEncoder** – Uses the scrypt hashing algorithm (memory-hard).
- **Argon2PasswordEncoder** – Uses Argon2 (modern and highly secure).

Why BCrypt is recommended

- It automatically **adds salt**, protecting against rainbow table attacks.
- It is **computationally expensive**, making brute-force attacks slower and passwords harder to crack.

👉 **In short**: BCrypt offers a strong balance of **security, performance, and wide industry support**, which is why it is commonly recommended.

7. With Spring Security 6 (Spring Boot 3+), how do you configure security without WebSecurityConfigurerAdapter?

In **Spring Security 6 (Spring Boot 3+)**, WebSecurityConfigurerAdapter is **removed**. Security is configured using **beans**, mainly SecurityFilterChain.

How to configure security now

- **Define a SecurityFilterChain bean**
  Configure HTTP security using the HttpSecurity DSL and return http.build().
- **Use lambda-based configuration**
  Authorization, CSRF, session management, login, and filters are configured using lambdas for better readability.
- **Define supporting beans**
  Such as PasswordEncoder, AuthenticationManager, and custom filters if required.

Example configuration

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults());

    return http.build();
}

@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

In short: Instead of extending WebSecurityConfigurerAdapter, you now **configure security using SecurityFilterChain and other explicit beans**, which is more modular and flexible.

8. **How do you secure endpoints in a Spring Boot application (URL-based rules)?**

In a Spring Boot application, **URL-based security** is configured using SecurityFilterChain and HttpSecurity.

Securing endpoints using URL-based rules

- **requestMatchers()**
  Used to define **URL patterns** and HTTP methods that security rules apply to.
  Example:

  requestMatchers("/api/public/")**

- **permitAll()**
  Allows **public access** to specific endpoints without authentication.
  Example:

  requestMatchers("/login", "/register").permitAll()

- **Role-based access**
  Restricts endpoints based on user roles using hasRole() or hasAuthority().
  Example:

  requestMatchers("/admin/").hasRole("ADMIN")**

  requestMatchers("/user/").hasAnyRole("USER", "ADMIN")**

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        );
    return http.build();
}
```

9. How do you enable method-level security, and what are the differences between @Secured, @PreAuthorize, and @RolesAllowed?

Enabling Method-Level Security (Spring Boot 3 / Spring Security 6)

To secure service or controller methods, enable method-level security using:

@EnableMethodSecurity

This annotation is added to a @Configuration class and replaces older annotations like @EnableGlobalMethodSecurity**.**

Differences between @Secured, @PreAuthorize, and @RolesAllowed

- **@Secured**
    - Supports role-based checks only
    - Uses ROLE_ prefix (e.g., ROLE_ADMIN)
    - Simple but less flexible

  @Secured("ROLE_ADMIN")
- **@PreAuthorize**
    - Most powerful and flexible
    - Uses SpEL expressions
    - Can check roles, permissions, method parameters, and custom logic

  @PreAuthorize("hasRole('ADMIN') and #id == authentication.principal.id")
- **@RolesAllowed**
    - Part of JSR-250 (jakarta.annotation)
    - Role-based only
    - Cleaner syntax, but limited features

  @RolesAllowed("ADMIN")

10. Explain how JWT authentication works in Spring Security:
Token creation, validation, and injection into the filter chain.

JWT Authentication in Spring Security

JWT authentication is commonly used for **stateless REST APIs**. It works through **token creation, validation, and integration into the Security Filter Chain**.

1️⃣ Token Creation (Authentication Phase)

- The user sends **username & password** to an authentication endpoint.
- Spring Security authenticates the user using AuthenticationManager.
- On successful authentication, a **JWT is generated** containing:
    - Subject (username/userId)
    - Roles/authorities
    - Issue time & expiration
- The token is signed using a **secret key or private key**.
- The JWT is returned to the client (usually in the response body).

2️⃣ Token Validation (Authorization Phase)

- The client sends the JWT in every request using:
- Authorization: Bearer <token>
- A **custom JWT filter** intercepts the request.
- The filter:
    - Extracts the token from the header
    - Verifies the **signature**
    - Checks **expiration**
    - Extracts user details & roles

3️⃣ Injection into the Security Filter Chain

- The JWT filter extends OncePerRequestFilter.

- It runs **before UsernamePasswordAuthenticationFilter**.
- If the token is valid:
    - A UsernamePasswordAuthenticationToken is created
    - It is stored in SecurityContextHolder
- Spring Security then treats the request as **authenticated**.

SecurityContextHolder.getContext().setAuthentication(authentication);

JWT Flow Summary

- **Create JWT** → on successful login
- **Send JWT** → in Authorization header
- **Validate JWT** → on every request
- **Set SecurityContext** → user is authenticated

👉 **In short**: JWT authentication makes Spring Security **stateless**, scalable, and ideal for modern REST and microservice architectures.

11. What is SecurityContextHolder?

SecurityContextHolder is a core class in Spring Security that **stores the security information of the currently authenticated user**.

What it contains

- Holds a SecurityContext
- SecurityContext contains the Authentication object
- Authentication stores:
    - Principal (user details)
    - Credentials
    - Authorities (roles/permissions)

How it works

- During authentication, Spring Security sets the Authentication object into the SecurityContextHolder.

- For every request, Spring Security reads user details from it to make authorization decisions.

```
Authentication auth = SecurityContextHolder.getContext().getAuthentication();
```

we can set it manually like this only when there custom security logic

```
SecurityContextHolder.getContext().setAuthentication(authentication);
```

Why it's important

- Provides **global access** to the current user's security data
- Enables role checks, method security, and access control

👉 **In short**: SecurityContextHolder is where Spring Security **keeps the logged-in user's identity and roles** during request processing.

10. **How do you implement OAuth2 login or resource server with Spring Security?**
Spring Security provides **first-class OAuth2 support** for both **OAuth2 Login (Authorization Code flow)** and **OAuth2 Resource Server (Client Credentials / JWT validation)**.

1️⃣ OAuth2 Login (Authorization Code Flow)

Used when:

- Users log in via **Google, GitHub, Keycloak, Okta, etc.**
- Browser-based applications

Configuration steps

1. Add dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

2. Configure client registration (application.yml)

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: xxx
            client-secret: yyy
            scope: openid, profile, email
```

3. Enable OAuth2 login

```java
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/login").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2Login(); // Authorization Code Flow
    return http.build();
}
```

Flow summary

- User is redirected to OAuth provider
- Provider authenticates user
- Authorization code is exchanged for an access token
- Spring Security creates an authenticated principal

2. OAuth2 Resource Server (Client Credentials / JWT)

Used when:

- Securing REST APIs
- Machine-to-machine communication
- Stateless microservices

Configuration steps

1. **Add dependency**

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

2. **Configure JWT validation**

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth-server.com/realms/demo
```

3. **Enable Resource Server**

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.jwt());
    return http.build();
}
```

Client Credentials Flow

- Client authenticates using client_id & client_secret
- Authorization server issues access token
- Client sends token in Authorization: Bearer
- Resource server validates token and scopes

In short:

- **Use OAuth2 Login for user authentication (Authorization Code flow)**
- **Use Resource Server for API protection (Client Credentials / JWT validation)**

12. What is session management in Spring Security and how do you configure it for stateless APIs?

Session management in Spring Security controls **how user authentication state is stored and maintained** between HTTP requests.

Session Management in Spring Security

- By default, Spring Security uses **HTTP sessions** to store the SecurityContext for logged-in users.
- This is suitable for **stateful, browser-based applications**.

Configuring Session Management for Stateless APIs

For **REST APIs using JWT**, sessions must be disabled to keep the application stateless.

Configuration

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session ->
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
        );
    return http.build();
}
```

What this configuration does

- STATELESS → Spring Security **does not create or use HTTP sessions**
- Authentication is **validated on every request** using JWT
- Improves **scalability** and suits microservices

Summary

- **Stateful apps** → Use sessions (default behavior)
- **Stateless APIs** → Disable sessions with SessionCreationPolicy.STATELESS

👉 **In short**: Stateless session management ensures Spring Security **does not store user state**, relying on tokens like JWT instead.

12. How do you implement custom authentication (e.g., custom filter or provider)?

Custom Authentication in Spring Security

What is Custom Authentication?

Custom authentication is used when **default Spring Security mechanisms (form login, basic auth)** are not sufficient. It allows developers to **define their own authentication logic** based on application requirements.

Why Do We Need Custom Authentication?

- **JWT / Token-based security**
  Default Spring Security uses sessions, but modern REST APIs require **stateless authentication using JWT**.
- **Non-standard credentials**
  Needed when authentication is based on **API keys, tokens, OTPs, mobile numbers, or headers** instead of username/password.
- **External system integration**
  Required when validating users from **LDAP, third-party services, legacy systems, or external auth servers**.
- **Custom business rules**
  To apply logic like **active/inactive users, device validation, MFA checks**, etc.
- **Microservices architecture**
  Each service must **independently validate tokens** without relying on sessions.

👉 **In short**: Custom authentication is required for **real-world, modern, and scalable applications**.

Ways to Implement Custom Authentication

1️⃣ **Custom Authentication Filter (Most Common – JWT)**

2️⃣ **Custom AuthenticationProvider.**

13. How do you write security tests for Spring Security endpoints?

Using @WebMvcTest, @WithMockUser, and mock authentication.

14. What are best practices to deploy Spring Security in production?

Best Practices to Deploy Spring Security in Production

---

1. Enforce HTTPS Everywhere

- Always use HTTPS in production.
- Protects credentials, tokens, and sensitive data from MITM attacks.
- Redirect all HTTP traffic to HTTPS.

---

2. Use Strong Password Hashing

- Never store plain-text passwords.
- Use **BCrypt** or **Argon2** for password hashing.
- BCrypt provides built-in salting and adaptive hashing.

---

3. Prefer Stateless Authentication

- Use **JWT or OAuth2** for REST APIs.
- Disable HTTP sessions in stateless applications.
- Improves scalability and avoids session-related attacks.

---

4. Secure Endpoints Explicitly

- Follow **deny-by-default** security principle.
- Public endpoints should be explicitly permitted.
- Protect sensitive endpoints using roles and authorities.

---

5. Configure CSRF Correctly

- Enable CSRF for browser-based applications (form login).
- Disable CSRF for stateless REST APIs using JWT.
- CSRF protection is not required when no session is used.

---

6. Enable Security HTTP Headers

- Protect against common web attacks.
- Enable headers for:
    - XSS protection
    - Clickjacking prevention
    - Content Security Policy (CSP)
    - Frame options

---

7. Protect Secrets and Credentials

- Never hardcode secrets in application.properties or code.
- Store secrets using:
    - Environment variables
    - Secret managers (Vault, AWS Secrets Manager)
- Rotate secrets regularly.

---

8. Use Method-Level Security

- Apply fine-grained access control using annotations.
- Common annotations:
    - @PreAuthorize
    - @Secured
    - @RolesAllowed
- Prevents unauthorized method access even if URL rules fail.

---

9. Handle Authentication & Authorization Errors Properly

- Return correct HTTP status codes:
    - 401 Unauthorized
    - 403 Forbidden
- Do not expose internal error details to clients.
- Customize exception handling where required.

---

10. Protect Against Brute Force Attacks

- Implement login attempt limits.
- Lock user accounts after repeated failures.
- Use CAPTCHA or rate limiting for public login APIs.

---

11. Follow JWT Security Best Practices

- Use short-lived access tokens.
- Implement refresh tokens.
- Send JWT only via **Authorization Header**.
- Never store tokens in plain text or expose them in URLs.

---

12. Enable Logging and Monitoring

- Log failed authentication and access attempts.
- Monitor suspicious activities.
- Integrate with centralized logging tools (ELK, CloudWatch).

---

13. Write Security Tests

- Test secured endpoints using mock users.
- Validate role-based access control.
- Prevent accidental exposure during future changes.

---

14. Keep Dependencies Up to Date

- Regularly update Spring Boot and Spring Security.
- Security patches are critical and frequent.
- Avoid using deprecated security configurations.

Short Interview Summary

In production, Spring Security should enforce HTTPS, use strong password hashing, follow stateless authentication where applicable, explicitly secure endpoints, protect secrets, enable security headers, and be continuously tested and monitored.

13. Why is SecurityContextHolder important and how is it propagated across threads?

SecurityContextHolder stores the **Authentication details of the currently logged-in user**. Spring Security uses it to perform authorization checks during request processing. By default, it uses **ThreadLocal**, meaning the security context is available **only within the same thread**. For async or multi-threaded execution, Spring provides strategies like **MODE_INHERITABLETHREADLOCAL** or explicitly passing the SecurityContext.

---

14. How does filter chain order affect security behavior?

Spring Security processes requests through a **chain of filters in a fixed order**.
If a filter (e.g., JWT filter) is placed **after** authorization filters, authentication may not happen in time, causing access denial.
Correct filter ordering ensures **authentication happens before authorization**, making filter order critical for proper security behavior.

---

15. User Roles vs Authorities in Spring Security

- **Role**: A special type of authority with ROLE_ prefix (e.g., ROLE_ADMIN)
- **Authority**: A general permission (e.g., READ_USERS, WRITE_USERS)

Spring internally treats **roles as authorities**, but roles are used for **coarse-grained access**, while authorities support **fine-grained permission control**.

---

16. Common Security Pitfalls in Spring Security Microservices (Short)

Common pitfalls include **sharing sessions across services**, improper **JWT validation**, missing **token expiration handling**, hardcoding secrets, lack of **centralized authentication**, and disabling security features like CSRF or CORS without understanding the impact. These issues can lead to **unauthorized access and token misuse**.